

Data Mining

Block 4 - Clustering & dimensionality reduction

Daniel Wlazło

SGH Warsaw School of Economics - SMMD-ADA / SMMD-AAB

27 October 2026

Today, hour by hour

- | | |
|--|---------|
| ① Recap - where the supervised half left us | ~5 min |
| ② The unsupervised mindset - no labels, new rules | ~15 min |
| ③ Distance & scaling - the choices before any algorithm | ~20 min |
| ④ K-means - and how to choose k | ~35 min |
| ⑤ Hierarchical & DBSCAN - two other shapes of “similar” | ~25 min |
| ⑥ PCA - compressing without a target | ~30 min |
| ⑦ Segmentation in practice - profiling, naming, traps | ~15 min |
| ⑧ Lab - notebook 04_clustering_pca | ~35 min |

Breaks roughly every 50 minutes.

Last week in five sentences

- 1 Trees turn data into **rules** and find interactions natively - but are too nervous to work alone.
- 2 Ensembles manufacture disagreement; every model family has a **leash**, and the U-curve is universal.
- 3 Judge by **CV mean $\pm$ spread**; rankings by **AUC/Gini**; probabilities by **calibration**; decisions by **expected cost**.
- 4 **Leakage** makes models look brilliant because every metric is its accomplice.
- 5 Complexity must **earn its keep** - and on the real portfolio it did, at every rung of the ladder.

All of that assumed one thing we are about to lose: **a label**.

The map, revisited

Paradigm	Question	Course status
Supervised	predict a known label	Blocks 2–3 ✓ (returns in 6)
Unsupervised	find structure, no label	Blocks 4–5 - now
Reinforcement	act, observe reward	mentioned only

- Within unsupervised, today covers the two classic families: **clustering** (group similar customers) and **dimensionality reduction** (compress features).
- Block 5 adds the other two: co-occurrence patterns and text.

What changes when the label disappears

Supervised (Blocks 2–3)

- “predict default”
- ground truth exists
- test set settles arguments
- errors have prices

Unsupervised (Blocks 4–5)

- “find **structure**”
- no ground truth at all
- no test set can say “correct”
- usefulness is judged by **decisions it improves**

The dangerous consequence: an unsupervised method **always returns an answer** - and nothing in the mathematics tells you whether the answer means anything. Scepticism graduates from virtue to method.

Why a lender clusters anyway

- **Customer segmentation** - strategy, product design, communication tone: “who are our customers, in four sentences?”
- **Anomaly detection** - customers unlike everyone else are interesting: fraud, errors, new markets (Block 1’s outlier perspective, upgraded).
- **A power tool for EDA** - clusters are hypotheses about heterogeneity you didn’t know to look for.
- **Compression** - hundreds of correlated features → a few components that feed supervised models (PCA, today).

What clustering is **not** for, in a regulated shop: making individual credit decisions. Segments inform strategy; the PD model prices the person.

The segmentation recipe

- 1 **Representation** - which features describe a customer? (application-time rule still applies!)
- 2 **Distance** - what does “similar” mean, numerically?
- 3 **Algorithm** - k-means / hierarchical / DBSCAN / ...
- 4 **Validation** - internal metrics, stability, **business sniff test**
- 5 **Naming** - from statistics to a story the organisation can use

Steps 1–2 decide more than step 3 - as usual, the thinking happens **before** the algorithm.

Evaluating without truth: the three substitutes

- 1 **Internal metrics** - do the clusters cohere numerically? (inertia, silhouette - today).
Necessary, never sufficient.
- 2 **Stability** - does the structure survive reseeding, resampling, next quarter's data? The unsupervised cousin of cross-validation.
- 3 **External usefulness** - profiled against outcomes held *out* of the clustering (default rate per segment), and the business sniff test.

Keep this triad in mind all day - every method we meet will be judged by it.

Distance is a modelling choice

For customers a, b with features x_a, x_b :

$$d_{\text{Euclid}}(a, b) = \sqrt{\sum_j (x_{aj} - x_{bj})^2} \quad d_{\text{Manhattan}}(a, b) = \sum_j |x_{aj} - x_{bj}|$$

- Euclidean: straight line - the default, sensitive to big single-feature gaps.
- Manhattan: sum of absolute gaps - more forgiving of one extreme coordinate.
- Cosine (angle, not length): the text-mining favourite - returns in Block 5.
- Mixed numeric + categorical data needs care: one-hot distances are crude; Gower's coefficient is the classical fix (mention, not exam material).

Worked example: how far apart are two customers?

Three standardised features, two customers:

	utilisation (SD)	log limit (SD)	age (SD)
Anna	+1.2	−0.5	−0.8
Bartek	+0.2	+0.7	−0.6
difference	1.0	−1.2	−0.2

$$d_{\text{Euclid}} = \sqrt{1.0^2 + 1.2^2 + 0.2^2} = \sqrt{2.48} \approx 1.57 \quad d_{\text{Manhattan}} = 1.0 + 1.2 + 0.2 = 2.4$$

- In standardised units, “1.57” means: about one-and-a-half typical customer-to-customer spreads apart.
- Note how the age gap (0.2) barely matters under Euclid - squaring amplifies the big gaps. That is a **choice** you made by picking the metric.

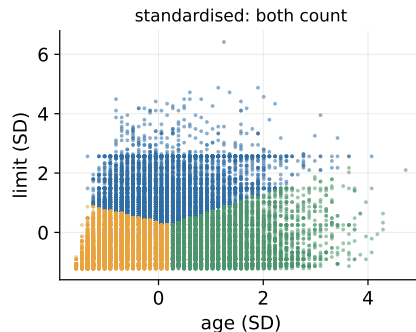
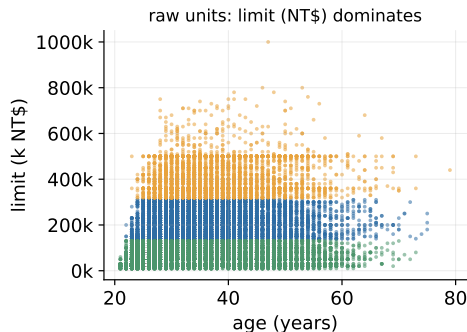
Which features deserve a vote?

The distance treats every column as one vote about similarity - so the feature list *is* the definition of “similar”.

- **Application-time rule still applies** - segments built on post-outcome fields inherit all of Block 3’s leakage pathologies.
- **One concept, one vote** - two utilisation variants let utilisation vote twice (the correlation heatmap knows).
- **Business-relevant only** - `customer_id` never; near-constant columns never (they vote “everyone is similar”).
- **Keep the target out** - `default` is reserved for *profiling*; clustering on it manufactures fake foresight.

Ours today: log limit, age, utilisation, months delayed, payment ratio, bill trend - six votes, one concept each. **Sex and marriage are excluded from the vote on purpose**: protected traits must not define “similar”, even in strategy work.

Scaling is not optional here

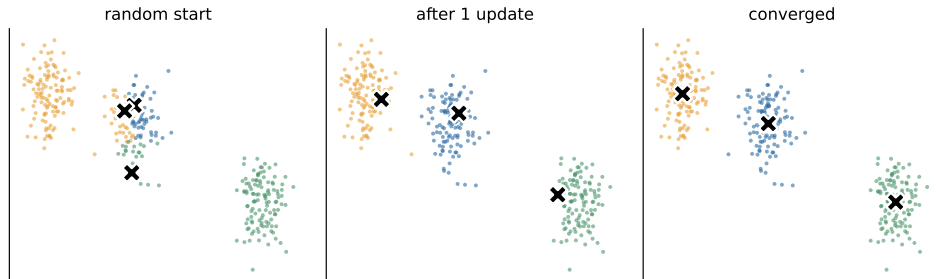


- Raw units: income (thousands of PLN) dwarfs age (tens of years) - distance **is** income, and the “clusters” are income bands.
- Standardise first, always. Trees forgave unscaled features (Block 3); **distances never do**.

The curse of dimensionality

- In high dimensions, distances **concentrate**: the nearest and the farthest customer become nearly equally far.
- Intuition: every extra feature adds a term to the sum; noise accumulates until everything is “average distance” from everything.
- Consequences for practice:
 - cluster on a **curated** feature set, not on everything you have;
 - correlated features silently **double-count** one concept (two utilisation variants = utilisation votes twice);
 - dimensionality reduction (PCA, in an hour) is often the opening move.

K-means in one loop



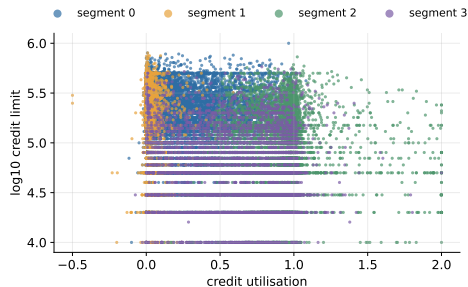
- Pick k centres; repeat: **assign** every point to its nearest centre, **move** each centre to its points' mean.
- Converges quickly - to a *local* optimum that depends on the start (hence `n_init`: run several starts, keep the best; k-means++ picks smarter starts).

What k-means optimises

$$\min_{\text{assignments}} \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2 \quad (\text{within-cluster sum of squares, "inertia"})$$

- Each customer belongs to the nearest centre $\Rightarrow$ the space is carved into **convex cells** around centroids.
- Baked-in assumptions: clusters are roughly **round**, similar in size and spread - in the *scaled* space.
- Squared distances: sensitive to outliers (a Block 1 reflex should fire - inspect before you cluster).

K-means on our portfolio

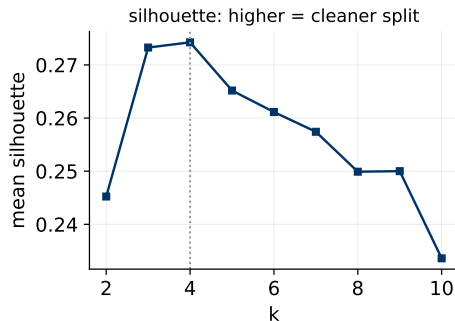
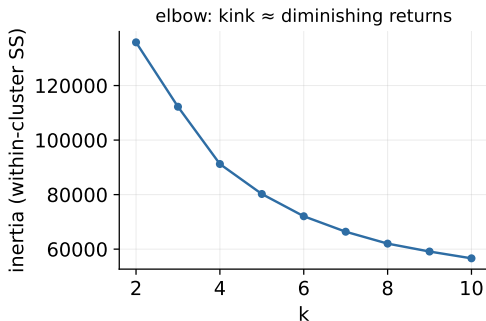


- Clustered on **six scaled features** (log limit, age, utilisation, months delayed, payment ratio, bill trend) - the plot shows just one 2-D shadow of it.
- No default column in the input - the target is reserved for *profiling* the segments later. (Using it would smuggle supervision in.)
- That dense wall of points at utilisation ≈ 0 ? **Transactors** - clients who pay in full every month. Real structure, visible before any algorithm runs.

Practical k-means: starts, speed, reproducibility

- **Local optima are real:** a bad random start yields a bad partition, silently. `n_init=10` runs ten starts and keeps the lowest inertia; **k-means++** spreads the initial centres out - both are cheap insurance.
- **Reproducibility:** the partition depends on the seed - `random_state` is part of the segmentation's recipe (Block 1's rule, unsupervised edition).
- **Scale:** k-means is fast - millions of rows are fine; `MiniBatchKMeans` trades a little quality for another order of magnitude.
- **Outliers:** squared distances make centroids chase extreme customers - inspect (or winsorise) first; or use k-medoids, which centres on actual customers.

Choosing k : elbow and silhouette



- **Elbow**: inertia always falls with k ; look for the kink of diminishing returns. Ours: smooth - **no clear kink**.
- **Silhouette** (-1 to 1): how much closer is each customer to its own cluster than to the next one? Ours **peaks at $k=4$** (0.27) - moderate, genuine grouping. (Block 1's synthetic cloud scored 0.12; same diagnostic, both verdicts honest.)

Silhouette by hand (exam-friendly)

For one customer i :

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

where a_i = mean distance to *own* cluster, b_i = mean distance to the *nearest other* cluster.

a_i	b_i	s_i	reading
0.4	1.6	$1.2/1.6 = 0.75$	deep inside its cluster
1.0	1.1	$0.1/1.1 = 0.09$	sitting on the border
1.5	0.9	$-0.6/1.5 = -0.40$	probably mis-assigned

- The *mean* silhouette over all customers is the number on the previous slide's right panel - our 0.27 now has a face: most customers sit clearly closer to home, with a genuine borderland in between.

Reading the diagnostics honestly

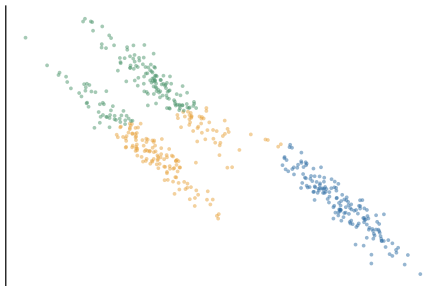
A silhouette of 0.27 is neither noise nor crystal. What does it mean?

- There **is** real grouping - the transactor wall and the delinquent tail are genuine behavioural islands - but the groups touch: many customers live near a border.
- Contrast with Block 1's synthetic portfolio (0.12, one cloud): the **same diagnostic** separated the two situations. That is what diagnostics are for.
- K-means returns exactly k sharp-edged segments **either way** - the number, not the picture, tells you how much to believe.

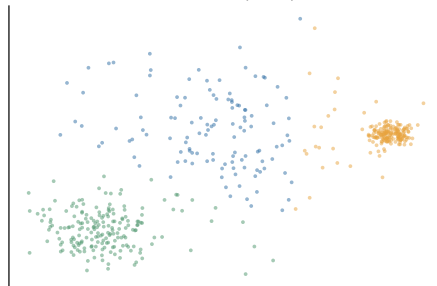
The report sentence survives: state k , the silhouette, and how firm the borders are - *every time*.

Where k-means breaks

k-means vs stretched clusters



k-means vs unequal spreads

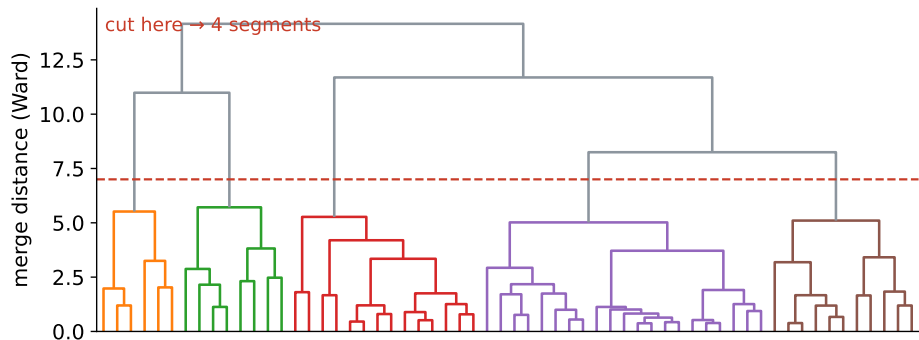


- Stretched or unequally spread clusters violate the “round and similar” assumption - centroids land in the wrong places **confidently**.
- If the shapes are wrong for the tool, change the tool - next section.

Agglomerative clustering: merge upward

- Start: every customer is its own cluster. Repeat: **merge the two closest clusters**. Stop when one remains - and record the whole merge history.
- “Closest” between *clusters* needs a definition - the **linkage**:
 - **single**: nearest members - chains, snakes;
 - **complete**: farthest members - compact, brittle;
 - **average**: in between;
 - **Ward**: merge that least increases within-cluster variance - k-means’ cousin, the usual default.
- Cost: pairwise distances - fine for thousands, painful for millions. (Sample first, or use k-means.)

The dendrogram: choose k *after* looking



- Height = distance at which two groups merged. Long vertical runs = well-separated groups; short ones = arbitrary cuts.
- One picture holds **every** k at once - cut where the jumps are big. (Ours merges early and often: the weak-structure verdict again, from a second witness.)

DBSCAN: clusters are dense places

- Forget centres. A cluster = a region where points are **packed**: core points (many neighbours within ϵ), their reachable friends - and everything else is **noise**.
- Two knobs: ϵ (how near is near) and `min_samples` (how many make “dense”).
- Three superpowers: finds **any shape**; decides the number of clusters itself; and **labels outliers** instead of forcing them into a segment.
- The price: knobs are touchy, and one global density - clusters of very different densities confuse it.

Fraud-detection perspective: DBSCAN’s “noise” bucket is sometimes the entire deliverable.

Choosing ε : the k-distance elbow

- DBSCAN's ε has its own diagnostic: for every point, compute the distance to its k -th nearest neighbour ($k = \text{min_samples}$); sort descending; plot.
- The curve falls steeply through sparse space, then flattens where density begins - **the knee is your ε** .
- Same philosophy as the elbow and the scree plot: let the data show its diminishing returns, cut there, and admit it is a judgement call.

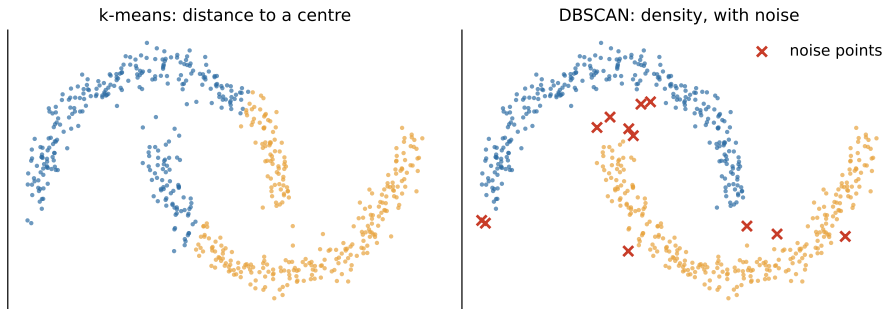
If no knee exists, densities vary too much for one global ε - consider HDBSCAN (hierarchical densities; mention only).

Soft memberships: Gaussian mixtures (a mention)

- K-means answers “which segment?”; a **Gaussian mixture model** answers “*with what probabilities?*” - every customer gets a membership vector, e.g. 70/25/5/0.
- Fitted by expectation–maximisation; k-means is the hard-edged special case (round, equal clusters, 0/1 memberships).
- Why credit likes it: borderline customers *are* the interesting ones, and a 55/45 split says “borderline” out loud - where k-means would silently pick a side.
- Same hygiene: scale first, choose components by BIC (its information-criterion cousin of the elbow), check stability.

sklearn: GaussianMixture - notebook extras.

Shapes k-means cannot see



- Two crescents: k-means slices them with a line (it must - convex cells); DBSCAN traces the density and quarantines the stragglers as noise.
- Crescents are rare in credit; the lesson is general: **the algorithm's assumptions decide what it can find.**

Choosing the clustering tool

	k-means	hierarchical	DBSCAN
Cluster shape	round, convex	any (linkage-dep.)	any dense shape
Chooses #clusters?	no (k input)	after (dendrogram)	yes
Outlier handling	forces them in	forces them in	noise label
Scales to millions	yes	no (pairwise)	middling
Randomness	init-dependent	none	none
First pick when	segments wanted	structure unknown	anomalies wanted

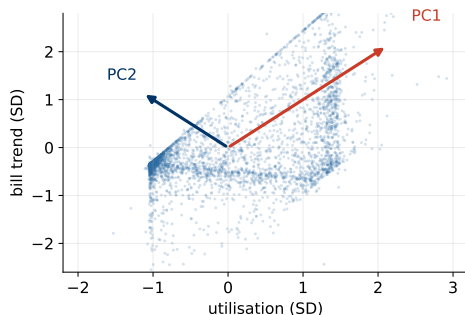
All three agree on one thing: **scale your features first.**

Why compress at all?

- **See** - humans plot in 2-D; portfolios live in 20-D. A faithful shadow beats twenty histograms.
- **Fight the curse** - fewer, decorrelated dimensions make distances meaningful again.
- **De-duplicate** - correlated features (Block 1's heatmap) carry one concept several times; components carry it once.
- **Feed models** - components can replace dozens of raw features in a supervised model (with a price: interpretability).

The workhorse: **principal component analysis** - linear, fast, half a century of service.

PCA: directions of maximal variance



- PC1 = the direction along which customers **differ most**; PC2 = the most-varying direction *perpendicular* to it; and so on.
- Here: income and savings co-vary (Block 1), so PC1 is their shared “affluence” axis; PC2 is what’s left.
- Each customer’s **score** on a component = their coordinate along that arrow. New features, built from old.

The machinery, in one breath

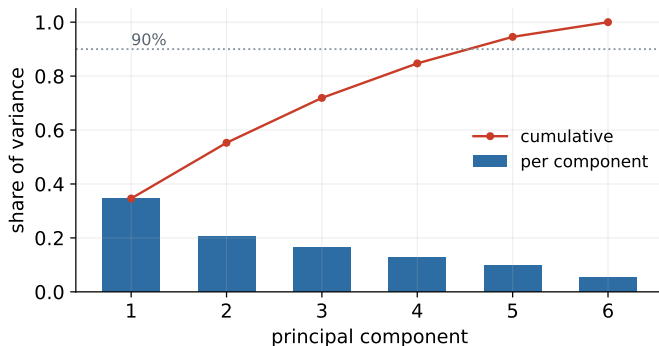
PC1 solves: find the unit direction with the most variance,

$$\max_{\|w\|=1} \text{Var}(Xw) = \max_{\|w\|=1} w^T \Sigma w \quad \implies \quad \Sigma w = \lambda w$$

- So the components are the **eigenvectors** of the correlation matrix Σ , ordered by their eigenvalues λ - which *are* the explained variances (computed via SVD in practice).
- Components are **orthogonal** (uncorrelated by construction) - multicollinearity dies here (Block 2's remedy #3, delivered).
- Each component is a **recipe**: a weighted mix of the original features. The weights are the **loadings** - and the key to interpretation.

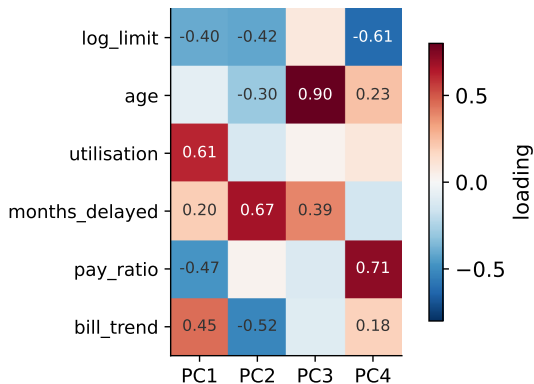
That is all the mathematics we need; sklearn: `PCA(n_components=...)`, `fit` / `transform` - the transformer API from Block 2.

How many components? The scree plot



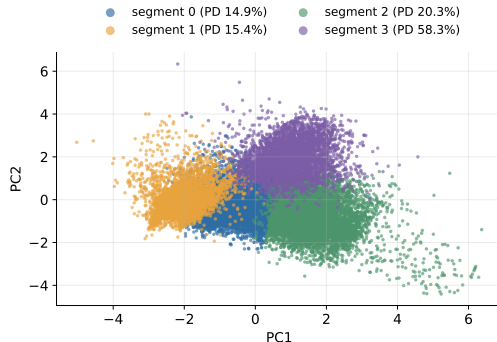
- Our portfolio: PC1 explains 35%, and 90% takes **five of six** components - moderate compression.
- The discount tracks the correlation structure exactly: the bill-driven features (utilisation, trend, payment ratio) share an axis; age stands alone. PCA compresses redundancy - **some redundancy, some discount**.

Reading the components: loadings



- PC1: +utilisation, +bill trend, –payment ratio, –limit - a **revolving-stress axis**. Nameable!
- Naming components is the same craft as naming segments - and just as necessary if anyone downstream is to trust them.
- A component nobody can name is a feature nobody can defend - remember adverse-action reasons.

Two views join: segments in PCA coordinates



- The 6-D segmentation, shadowed onto the first two components: the transactors and the distressed pull visibly apart along **PC1, the revolving-stress axis** - the silhouette's verdict, drawn.
- And the legend carries the punchline: default rates from 15% to **58%** - structure a strategy team can act on.

Another lens: PCA as compression with a receipt

- Keep $m < p$ components, then *decompress*: rebuild each customer from their m scores. The rebuild is imperfect -

$$\text{reconstruction error} = \sum_i \|x_i - \hat{x}_i\|^2 = \text{variance you threw away.}$$

- “Explained variance 90% with 5 components” and “reconstruction loses 10%” are the **same sentence** - use whichever the audience hears better.
- This lens generalises: autoencoders (Block 6’s cousins) are exactly this idea with nonlinear compress/decompress.

Components as features - the leakage-safe way

- Tempting move: replace the six features with the top components and feed them to the Block 3 tournament.
- The golden rule follows: PCA has fitted parameters (the loadings) - **fit on training data only**, inside the pipeline:

```
make_pipeline(StandardScaler(), PCA(5), LogisticRegression())
```

- Inside cross-validation, the loadings then refit per fold - automatically. Outside a pipeline, they silently leak.
- Expect no miracle on our portfolio (little redundancy to compress) - the point is the *pattern*, which pays off on wide, correlated real data.

PCA: the fine print

- **Scale first** - unscaled, PC1 is simply the biggest-variance feature in its raw units (income, again).
- **Variance $\neq$ importance** - PCA never saw the target; the risk signal may hide in PC6. Dropping small components can drop the signal (check against the supervised task before feeding components to models).
- **Linear only** - PCA finds flat directions; curved structure survives compression badly.
- For *visualisation* of curved structure: t-SNE and UMAP - beautiful, nonlinear, and **treacherous**: distances between their islands and their densities mean little. Use for pictures, never for downstream distances.

Profiling the segments (ours, $k = 4$)

Segment	size	limit	util.	delays	default rate
0 "mainstream revolvers"	12,562	174k	0.20	0.3	14.9%
1 "transactors" (pay in full)	4,980	174k	0.03	0.3	15.4%
2 "maxed-out borrowers"	8,576	72k	0.90	0.4	20.3%
3 " distressed delinquents "	3,882	54k	0.60	4.2	58.3%

- Profile = per-segment means of the *clustering* features **plus** outcomes held out of the clustering (default rate!) - that is where the business value appears.
- Segment 3: four months behind on average, defaulting at **58%** - over 2.5× the portfolio's 22%. The story writes itself - and this is the classic industry quartet (transactors / revolvers / maxed-out / distressed), rediscovered from scratch by an unsupervised algorithm.

From statistics to a story: naming

- A segment name must be earned by the profile table: short, concrete, slightly memorable - “stretched borrowers”, not “segment 1” and not “high-DTI-low-income cluster”.
- The test: a product manager should predict the profile *from the name alone*.
- Dishonest naming is a real failure mode: names imply crisp tribes where the data shows a continuum. Pair every name with the **soft-boundaries disclaimer** from the silhouette slide.
- And write down the recipe (features, scaling, k , seed) - Block 1’s reproducibility rule; next quarter’s refresh must rebuild *these* segments, not new ones.

Is the segmentation stable?

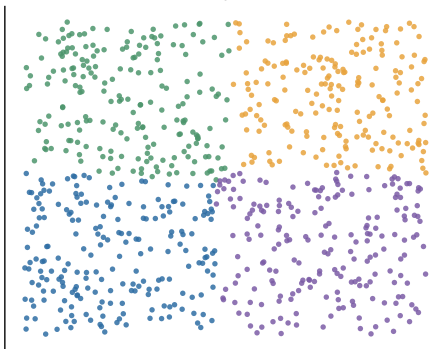
With no ground truth, stability is the closest thing to correctness:

- **Reseed**: different random starts - same segments? (Ours: yes, with `n_init=10`.)
- **Resample**: cluster on 80% subsamples - do customers stay together? (The unsupervised cousin of CV.)
- **Re-time**: next quarter's data - do the segments reappear, or did we photograph noise?
- **Perturb**: drop one feature - does the story survive, or did one column dictate everything (the unscaled-income disease in disguise)?

Then the strongest validator available: **does the business recognise these customers?**

The sobering slide

k-means, $k=4$, on pure uniform noise



- Pure uniform noise. No structure whatsoever. K-means, asked for four clusters, **delivers four clusters** - tidy, colourful, publishable.
- Every clustering deliverable should survive this question: *“what would this slide look like on noise?”*
- The unsupervised twin of Block 1’s “torturing the data until it confesses”.

Bonus recipe: k-means as an anomaly detector

- Every customer has a **distance to their nearest centroid** - a one-line outlier score: the further, the stranger.
- Shortlist the top 1% and hand it to investigators - a serviceable first fraud screen with zero labels required.
- DBSCAN offers the same for free: its **noise bucket** is the shortlist.
- Block 1's lesson upgraded: outliers are not cleaning waste - sometimes the strangest customers *are* the deliverable.

Notebook extras: build the shortlist and eyeball its profile against the portfolio.

Common segmentation sins

- ❶ **Clustering on the target** (or its twins) - manufactured foresight; Block 3's leakage in a new costume.
- ❷ **Unscaled features** - one column secretly dictates the whole segmentation.
- ❸ **Too many segments** - fourteen segments no one can name is a lookup table, not insight.
- ❹ **Hard-boundary storytelling** - crisp names on a continuum, without the soft-boundaries sentence.
- ❺ **Quarterly renaming** - refitting from scratch each quarter and re-deriving "new" customer types from seed noise.
- ❻ **Skipping the noise question** - never asking what the deliverable would look like on shuffled data.

Where segments are allowed to work

- **Strategy & product:** design offers per segment; watch segment sizes drift over quarters (a leading indicator of portfolio change - ties to Block 3's monitoring).
- **Communication:** tone and channel per segment.
- **As features:** segment membership can enter a supervised model - fit the clustering on **training data only** (the golden rule follows us everywhere).
- **Not:** as the individual credit decision. Pricing a *person* by their *segment* invites both regulatory trouble and unfairness - the PD model prices the person (and Block 6 returns to fairness properly).

Laboratory

Notebook 04_clustering_pca - laptops out.

Lab today

Notebook 04_clustering_pca

- scale, cluster, and *break* it (unscaled demo)
- elbow + silhouette - read weak signals honestly
- profile and **name** the $k = 4$ segments
- dendrogram on a sample; DBSCAN on shapes
- PCA: scree, loadings, segments in 2-D

Done =

- a named segment table with sizes, profiles, and default rates
- one honest paragraph on how strong the structure really is

Optional extras

- DBSCAN's noise bucket as an anomaly shortlist
- components as features in the Block 3 tournament

Summary

What to take home from Block 4.

Block 4 in five sentences

- 1 Without labels there is no “correct” - only **useful**, and usefulness is argued, not measured.
- 2 **Distance is a modelling choice**, and unscaled features make it silently for you - always standardise.
- 3 K-means carves round cells and **always answers**; elbow, silhouette and stability tell you how much to believe it.
- 4 Hierarchies show all k at once; **DBSCAN** finds shapes and names outliers - assumptions decide what each tool can see.
- 5 **PCA** compresses redundancy into nameable axes - and its discount always equals the redundancy it finds - no more, no less.

Project status & reminder

- Three weeks to the presentation (Block 7) - your two model families should exist **this week**, however rough, so the final fortnight belongs to evaluation and the slides.
- If your project uses segmentation: it must arrive with the honesty kit - silhouette or stability evidence, the soft-boundaries sentence, and named profiles. A pretty scatter plot is not a deliverable.
- Rubric reminder: interpretation & recommendation is 15% - segment naming and profiling is precisely that skill.

Office-hour priority this week: teams whose topic description flagged unresolved data-quality risks.

Exam practice - multiple choice

Q1. Why must features be standardised before k-means?

- ☐ a) k-means only converges on standardised data
- ☐ b) features with large units dominate the distance, deciding “similar” by themselves
- ☐ c) standardisation increases the silhouette score
- ☐ d) scikit-learn requires it

Q2. Your segmentation reports a mean silhouette of 0.27 at $k = 4$. The honest reading:

- ☐ a) the clustering failed - do not report it
- ☐ b) moderate, real-but-touching groups - report the number alongside the segments
- ☐ c) excellent separation - the segments are firm
- ☐ d) k must be increased until silhouette exceeds 0.5

Exam practice - open question

In the style of the term-0 exam (~60% open questions)

A PCA of six behavioural features yields PC1 loadings: +utilisation, +bill_trend, -pay_ratio, -log_limit. Name this component for a product manager and justify the name from the loadings. What business action could be indexed to it?

4-6 sentences. A component you can name is a component you can defend.

Next week: pattern discovery & text mining.

What sits in the basket - and what sits in the complaints inbox.

Keep learning - Block 4 reading

- **Jain** - *Data Clustering: 50 Years Beyond K-means* (Pattern Recognition Letters, 2010) - the survey that puts today's toolbox in one story.
- **Jolliffe & Cadima** - *Principal component analysis: a review and recent developments* (Phil. Trans. R. Soc. A, 2016).
- **Wattenberg, Viégas & Johnson** - *How to Use t-SNE Effectively* (distill.pub, 2016) - interactive; why 2-D embeddings seduce and mislead.
- **scikit-learn User Guide** - *Clustering* section: the algorithm-comparison gallery is worth ten lectures.